

An approach to the mapping problem

Carolina Baltazar-107117 Beatriz Coelho-107113 Eduardo Larginho-103358 Igor Paruque-102901
Instituto Superior Técnico, Lisboa

Abstract—This paper addresses the mapping problem in autonomous systems using the occupancy grid mapping algorithm. The goal is to understand how a map can be created as accurately as possible with error-prone sensor data, while knowing where the robot is in an unknown or outdated environment. Throughout this article, a theoretical analysis as well as multiple attempts to map a corridor with a robot and a range sensor will show what factors have an impact on the performance of this mapping algorithm. Afterwards, the comparison between the created maps will be explained and utilized to understand the best way to generate or update a map.

Index Terms—occupancy grid mapping, log odds, odometry, cell, map

I. INTRODUCTION AND MOTIVATION

The importance of mapping with known location in robotics lies in the necessity to know, with enough detail, how to move from one location to another in the most efficient way possible, specially when the locations in question and the space in between them are constantly changing. It is important that their representation keeps up with those changes to maximize the map's value. An example where the location of an object is used to figure out what's around it is the update of a room's map that autonomous vacuum cleaners like the Roomba perform when a chair is displaced a few inches in any direction. After using vSLAM to map an entire room, they use that information to make such small adjustments when needed, avoiding an unnecessary and inefficient remap process.

Unlike the SLAM (Simultaneous Localization And Mapping) problem, the location of the robot inside the environment is always known, shifting the focus to figuring out what the environment looks like. At first glance, it would feel like a much easier problem to deal with, since half of the SLAM problem is solved by a good localization algorithm. However, this problem still has its hurdles to overcome. These hurdles include i) the map's size, ii) the existence of cycles, iii) the errors in perception and actuation, iv) the perceptual ambiguity and v) moving objects.

The following sections of this paper will describe one of the ways to map an environment in theory; how to predict real world behavior in a simulator; the setup used to physically attempt to map a space; how close the simulations and the real-world attempts are to each other; and finally the aspects that truly dictate the accuracy of the map.

II. METHODS AND ALGORITHMS

This approach to the mapping problem will be done with the help of a Pioneer 3DX. This sturdy robot will serve as a carrier

for our range sensor. From here on out, two assumptions will be made: 1) Only the center of mass of the robot will be taken into account and 2) The position and orientation of the robot in the environment is always known. From there, we can use the following figure to understand what happens during the execution of our mapping algorithm:

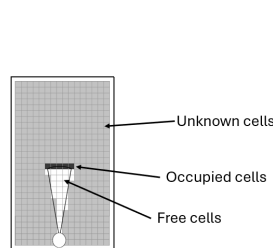


Fig. 1. Visualization of what the occupancy grid mapping looks like.

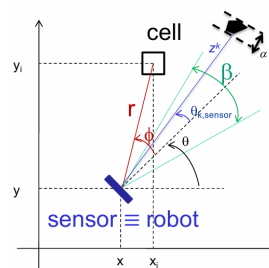


Fig. 2. Description of the physical variables involved in the mapping process. We assume the sensor position corresponds to the Pioneer's center of mass.

Looking at Fig. 1, we can see that occupancy grid mapping starts by considering a finite section of the environment and divides it into square cells where the length of its side will determine the resolution of our map. Each cell will have three possible states: The cell starts with an unknown state and can either transition to a free or occupied state. From the distance measurements of the range sensor, this algorithm will compute how likely a given cell is to be free or occupied, while the unknown state is for cells that were not scanned yet. This paper will make use of Fig. 2 as the starting point to analyze what happens between the sensor and one of the map's cells.

In this figure, r is the euclidean distance between the sensor and the cell to be scanned, θ is the orientation of the sensor, β is the sensor's field of view, z^k is the distance between the sensor and an obstacle measured at an instant k by the sensor, α is the thickness of the obstacle, ϕ is the difference between θ and the angle of r , $\theta_{k,sensor}$ is the difference between θ and the angle of z^k , $x_i - x$ is the distance on the x axis between the sensor and the cell to be scanned and $y_i - y$ is the distance on the y axis between the sensor and that same cell. In the following bullet points that range from A to F are a brief description of the concepts used in this mapping technique.

A. Sensors used

Laser – Hokuyo URG-04LX-UG01 Laser LIDAR sensor mounted on the Pioneer. This sensor provides horizontal

distance measurements over a 240° field of view. Each scan returns a series of distance readings, representing the distances from the robot to obstacles in its environment, with a maximum range of 4m, and minimum range of 2cm. This data is being published in real time on the ROS topic `/scan`.

Odometry – Process of estimating a change in position over time through sensor data. The Pioneer provides odometry data through wheel encoders on its differential drive. This allows the robot to estimate its position and orientation over time. The data is published on the `/odom` topic.

B. Occupancy Grid Mapping

Occupancy Grid Mapping is a probabilistic method used to represent the environment of a mobile robot. It divides the space into a grid of cells, where each cell stores the probability of being occupied, free or unknown. The main components implemented in our system include the Bresenham Line Algorithm, and the Log-Odds Update method.

C. Bresenham's Line Algorithm

To determine which cells are traversed by a laser beam, we use Bresenham's Line Algorithm. This algorithm computes which discrete grid cells lie along a straight line between two points. In our case, it is used to draw a line from the robot's current position to the endpoint of a laser measurement. All cells along this line (except the final one) are considered free, while the last cell is marked as occupied. This technique ensures that the laser path is accurately represented in the grid.

D. Log-Odds Update

To probabilistically represent occupancy, we use the log-odds formulation, which is more suitable for iterative Bayesian updates. Each cell in the map maintains a log-odds value, initialized at 0 (indicating an unknown state). When a laser beam passes through a cell, we decrease its log-odds value (-0.4), increasing our belief that the cell is free. When a laser hits an obstacle in a cell, the log-odds is increased (+0.85), reflecting a higher probability of occupancy. Multiple consistent readings over time strengthen the confidence in each cell's classification.

Mathematically, the update for a cell c at time t is:

$$\ell_t(c) = \ell_{t-1}(c) + \log \left(\frac{p(c | z_t)}{1 - p(c | z_t)} \right) - \log \left(\frac{p(c)}{1 - p(c)} \right)$$

III. IMPLEMENTATION

A. Code

With the aforementioned descriptions of variables and concepts, we can discuss how the occupancy grid mapping algorithm was implemented in this block of pseudo-code:

Algorithm 1 Occupancy Grid Mapping

```

1: Initialize:
2: Define map size and resolution
3: Set all log-odds cells to 0
4: Initialize robot pose  $(x, y, \theta) = (0, 0, 0)$ 
5: while ROS is running do
6:   Pose Estimation:
7:   Obtain transform from odom to base_link
8:   if transform is unavailable then
9:     Skip iteration
10:  end if
11:  Laser Processing:
12:  for each valid range  $r$  in LaserScan do
13:    Compute angle  $\phi = \text{angle\_min} + i \cdot$ 
    angle_increment
14:    Compute hit point:
15:     $x_{\text{end}} = x + r \cdot \cos(\theta + \phi)$ 
16:     $y_{\text{end}} = y + r \cdot \sin(\theta + \phi)$ 
17:    Convert robot and end point to grid indices
     $(x_0, y_0), (x_1, y_1)$ 
18:    Run Bresenham's algorithm from  $(x_0, y_0)$  to
     $(x_1, y_1)$ 
19:    for each intermediate cell  $(x, y)$  do
20:       $\log\_odds[x][y] = \log\_odds[x][y] - 0.4$ 
21:      Clip  $\log\_odds[x][y]$  to  $[-10, 10]$ 
22:    end for
23:    At final cell  $(x_1, y_1)$ :
24:     $\log\_odds[x_1][y_1] = \log\_odds[x_1][y_1] + 0.85$ 
25:    Clip to  $[-10, 10]$ 
26:  end for
27:  Grid Publishing:
28:  for each cell  $(x, y)$  do
29:    Compute probability  $p = 1 - \frac{1}{1 + e^{\log\_odds[x][y]}}$ 
30:    if  $p < 0.01$  then
31:      Mark cell as free (0)
32:    else if  $p > 0.97$  then
33:      Mark cell as occupied (100)
34:    else
35:      Mark cell as unknown (-1)
36:    end if
37:  end for
38:  Publish map on topic /my_map
39:  Optionally save map image every 10 seconds
40: end while

```

The robot starts in the unknown and we consider its initial pose as the reference point for whole map. For every point in space the robot moves to, we can choose to estimate its new pose with odometry data or with AMCL data. After getting the pose, the algorithm will process all the distance measurements inside the sensor's range and field of view to estimate the state of the cells in between the obstacles that were within range and the cells at the obstacles' point in space. These estimated states come in the form of log-odds to make their variation more stable and will later be published to the map as probabilities with the logistic function. The resulting probabilities will tell

how likely each cell is to be occupied. If a cell's probability is bigger than 0.97, that cell is considered as an occupied cell and will be colored black. If it's smaller than 0.01, the cell is considered a free cell and will be colored white. Any other values for the probabilities will force the cell to remain in an unknown state with a gray color. This process happens in a loop until the robot covers all the necessary points in the environment to map it completely.

B. Pose Estimation

In order to know the pioneer pose we used two approaches:

Odometry

Transform between the /odom and /baselink frames using the TF system in ROS. This provides a continuously updated estimate of the robot's position (x, y) and orientation θ by integrating wheel encoder data. While this model is simple and effective, it accumulates error over time due to factors such as wheel slip and sensor noise.

Odometry Evaluation To evaluate the limitations of odometry for mapping, we conducted a controlled test where the Pioneer 3-DX robot, equipped with a Hokuyo laser scanner, was driven along a nearly perfect 1-meter square trajectory. We visualized the estimated path using a simple script that subscribed to the /pose topic.

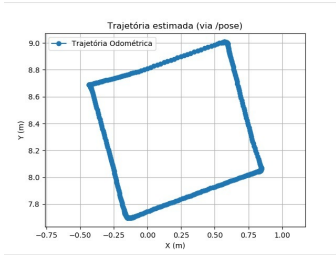


Fig. 3. Estimated trajectory on a 1m square path (calibrated odometry)

In this scenario, the trajectory closed almost perfectly, confirming that the calibrated odometry was sufficient to eliminate most of the systematic drift over short distances. However, when applying the same calibration in the 5th floor of the North Tower, we observed that the square trajectories did not close as precisely. This is expected, as the corridors on that floor form a square with sides of approximately 20 meters. Over such a long distance, even small residual calibration errors accumulate significantly, leading to noticeable deviation and preventing perfect loop closure.

As shown in the resulting figure, we first measured the angles of the square to check whether they were close to 90 degrees. They were not, confirming the presence of angular drift. This type of error was also noticeable in the maps generated using odometry alone. Despite this, the robot's estimation of distances was fairly accurate: each side of the square trajectory measured approximately 1 meter, indicating that

the translational motion was well estimated, while rotational motion showed systematic error.

During the mapping process, we noticed that the robot's path would not close properly when completing loops, particularly square ones. This pointed to unbalanced odometry, causing errors to accumulate and resulting in distorted maps.

To mitigate this, we adjusted the odometry calibration parameters exposed by the p2os driver package: left motor gain and right motor gain. These values control the relative power applied to the left and right motors, enabling us to correct asymmetries in the robot's movement.

Our initial tests used the default values (1.0 for both gains), which resulted in a consistent drift to the right. We then tried increasing the left gain and decreasing the right gain (left motor gain=1.05, right motor gain=0.95), but this only worsened the deviation, suggesting that the correction was applied in the wrong direction.

Through iterative testing, running the robot along square paths and visually comparing the resulting maps, we eventually found a configuration that significantly improved accuracy: left motor gain=0.8 and right motor gain=1.2. With these values, the robot's path was much more stable and symmetrical, and the generated map was notably more accurate and consistent.

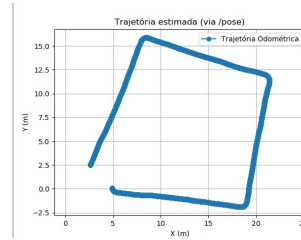


Fig. 4. Trajectory of the Pioneer without odometry calibration

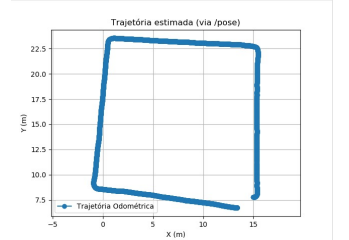


Fig. 5. Trajectory of the Pioneer with calibrated odometry

Figure 3 shows the trajectory of the Pioneer throughout the 5th floor of the North Tower, using the default, uncalibrated odometry settings, while Figure 4 illustrates the trajectory after odometry calibration. We can observe a noticeable improvement: the distance between the final and initial positions is significantly reduced, indicating better trajectory closure. This confirms that the calibration process improved the accuracy of the robot's motion estimation. Additionally, the overall trajectory appears more consistent with the intended path, exhibiting less unwanted curvature and closer alignment with the robot's actual motion.

We documented the mapping results before and after odometry calibration, and the improvement was evident. This calibration step was essential in improving the quality of the mapping. It shows how critical odometry tuning can be for reliable 2D environment representation.

AMCL

As an attempt to correct the errors that come with using only odometry, we decided to use AMCL to obtain the pose of the robot. Since the package needs a map figure, we decided to run the ROS package GMapping – it proved to be useful not only for the localization with AMCL but also to use as the ground truth to evaluate our performance. AMCL provides the transform between the *map* and *base_link* frames using the TF system in ROS. It estimates the robot’s pose (x, y, θ) on a known map by applying a particle filter to sensor data and motion updates. Unlike pure odometry, AMCL corrects for accumulated drift by continuously comparing observed sensor readings to the map.

However, just running AMCL proved to be more challenging than expected. We noticed that our bag, midway through, introduced TF inconsistencies that caused sudden and unrealistic jumps in the estimated robot pose. These discontinuities disrupted the mapping process by corrupting the alignment of LaserScan data with the robot’s position. To mitigate this, we added a fallback to use the most recent transform. We also updated our code to fetch the robot’s pose at the exact timestamp of each incoming LaserScan, since AMCL uses scan matching with a static map and the pose estimates may change abruptly.

Despite not being able to generate a map with all four corridors, AMCL succeeded in doing so with two and even three corridors in some runs. The main slight improvement was that the angles between the corridors were closer to 90° , indicating better orientation estimates compared to the odometry-only version, where the corridors were noticeably skewed.

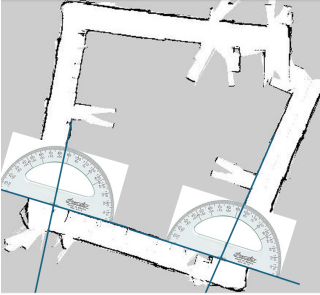


Fig. 6. Map generated using our mapping code

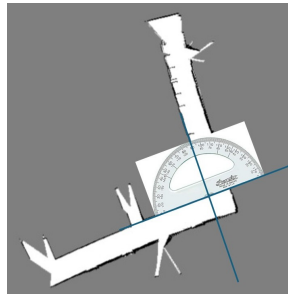


Fig. 7. Map generated using AMCL for localization

As observed in Figures 5 and 6, the angles of the corners generated using AMCL are closer to the real values. Specifically, Figure 5, which was generated using our code with uncalibrated odometry, shows a corner angle of approximately 95° , while Figure 6, which uses AMCL for pose estimation, presents a more accurate angle of 90° . This comparison was made by placing a protractor at the corners of each image, as shown in the figures. From this analysis, we can see that AMCL improves the accuracy of angular estimation; however, it does not necessarily lead to better overall map quality.

IV. MAPPING SIMULATOR

To validate the performance of our occupancy grid mapping algorithm under realistic conditions, we developed a custom simulation environment in Python using the ‘*pygame*’ and ‘*OpenCV*’ libraries. The simulator recreates a mobile robot equipped with odometry and a *Hokuyo*-like planar laser range finder. While we approximate the physical behavior of the *Hokuyo* laser scanner, our simulated device replicates its essential operational characteristics. Specifically, it performs a sweeping scan over a 150° field of view, emitting laser beams at 1° angular resolution, resulting in 151 individual rays per scan. Each beam is independently processed to simulate realistic range measurements, with a maximum sensing distance of 4 meters. To better reflect real-world sensor imperfections, the simulation includes Gaussian noise, probabilistic false negatives (missed detections), and occasional false positives (spurious readings), to reproduce the imperfections that researchers commonly encounter in real-world range data.

A. Ground Truth Environment Design

Our environment relies on a grayscale occupancy map where black regions represent obstacles and white regions correspond to free space. Importantly, the robot never has access to this map during simulation. Instead, the map serves exclusively as our ground truth reference, enabling us to quantitatively evaluate how well the robot builds its internal representation. This design choice reflects real-world scenarios where robots must construct their understanding of the environment from scratch, depending entirely on their sensors and motion estimates.

B. Sensor Simulation and Data Processing

When we initialize the system, the robot begins in a randomly selected free-space region, which we detect through thresholding. From this starting position, the simulated *Hokuyo* range finder performs periodic scans, generating beams that we project into the environment using *Bresenham’s* line algorithm. This algorithm helps us identify which grid cells each beam traverses and distinguishes between free space and potential obstacles. We treat the final cell along each beam’s path as a possible obstacle hit, while assuming intermediate cells represent free space. We then process these observations using an inverse sensor model to produce log-odds updates for the occupancy grid.

C. Occupancy Grid Mapping Implementation

Each grid cell maintains a log-odds value that reflects its probability of being occupied. We weight updates to these values based on spatial consistency and proximity, aiming to improve robustness against noise and better model uncertainty over time. Cells that we consistently observe as free or occupied gradually develop high-confidence values, while conflicting observations result in intermediate values that indicate uncertainty. The robot’s internal map therefore

evolves incrementally as it collects new scan data, effectively simulating the Bayesian nature of real occupancy mapping.

D. Pose Uncertainty Modeling

Alongside the mapping process, our simulator models pose uncertainty by introducing noise into the robot's odometry. The true position evolves deterministically, while the estimated position accumulates drift through both systematic bias and stochastic error. We maintain a covariance matrix to model this uncertainty and visualize it graphically as an ellipse centered at the estimated position. This uncertainty - aware motion model proves critical for evaluating how mapping performance degrades under localization error - a frequent challenge that autonomous systems face in practice.

E. User Interface and Visualization

Our graphical interface provides real-time visualization of the occupancy grid, sensor rays, true and estimated robot trajectories. We control the robot manually through keyboard input, with options for adjusting speed, resetting the simulation, and exporting the final map. Crucially, all mapping decisions rely solely on the robot's own perception and motion model, ensuring that our simulation respects the constraints that real-world robotic systems must navigate.

F. Simulation Results

To evaluate the mapping algorithm, we provide both quantitative and qualitative results. Overall the simulated results with a low-noise had a IoU of 83% and the ones with high-noise had an IoU of 46%,

The simulated and experimental results are represented in the Table II, in the experimental results section summarizes the mapping accuracy using the (IoU) metric across different configurations.

Additionally, the following figures show simulated occupancy maps, visually illustrating the impact of noise and odometry uncertainty on the final map quality.

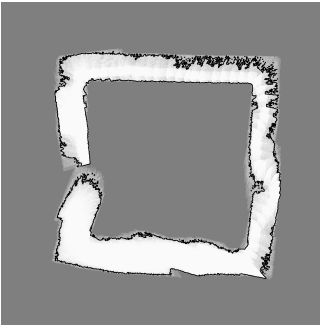


Fig. 8. Simulation map - with noise

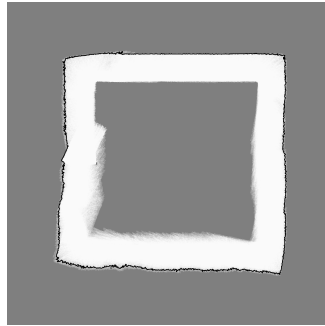


Fig. 9. Simulation map - almost 0 noise

Following the simulator images, it becomes clear that simulation plays a crucial role in understanding and validating the performance of mapping algorithms under controlled and repeatable conditions.

The contrast between low-noise and high-noise scenarios highlights how sensor inaccuracies and pose uncertainty can significantly degrade mapping quality.

By incorporating realistic noise models and visualizing both estimated and ground truth trajectories, the simulator offered valuable insights into the robustness and limitations of our approach.

V. EXPERIMENTAL RESULT

All experimental data used in this work were collected on the 5th floor of the North Tower at Instituto Superior Técnico. The mapping was performed throughout the entire floor, covering all corridors, using the Pioneer 3DX equipped with a Hokuyo laser. During data collection, we recorded basic ROS bag files containing only raw sensor data, namely LaserScan and Odometry topics, without running any mapping packages such as GMapping during runtime. These datasets were later used to evaluate our mapping implementation and test the use of AMCL for localization. .

A. Mapping using AMCL, for pose

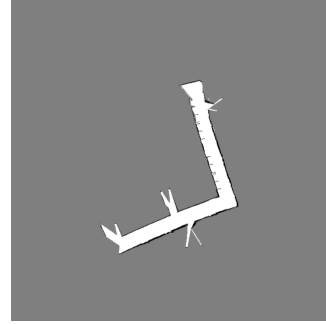


Fig. 10. Mapping using AMCL 1

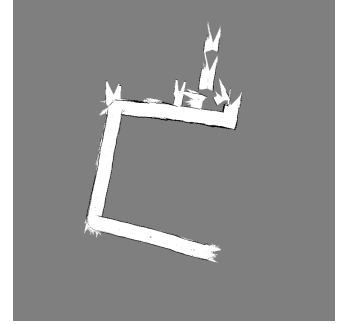


Fig. 11. Mapping using AMCL 2

B. Mapping using odometry for pose

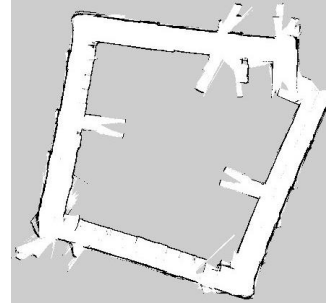


Fig. 12. Map generated using uncalibrated odometry

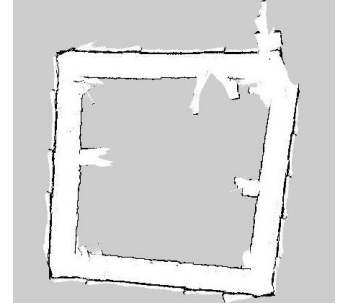


Fig. 13. Map generated using calibrated odometry

C. Quality of the map

As mentioned earlier, we generated a map using GMapping, not only to serve as a reference for AMCL but also to act as ground truth for comparison with our mapping results.

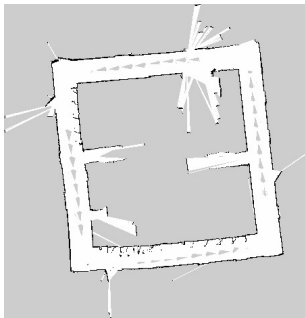


Fig. 14. Map generated using GMapping – Ground truth

Before performing the comparison, we ensured that both maps (GMapping and ours) had the same dimensions, resolution and colours (since GMapping had slightly different colours compared with our initial map). The alignment was achieved by resized the GMapping-generated map to 50x50 pixels to match our map's resolution and by updating the .yaml.

After these adjustments, we proceed to create a map that highlights the areas where our map matches the one produced by Gmapping. The code identifies which cells are valid in both maps (cells that are not -1), checks where they match, and highlights those cells in white, leaving the rest of the map black, meaning the cells common to both maps appear white and mismatched or unknown areas appear black.

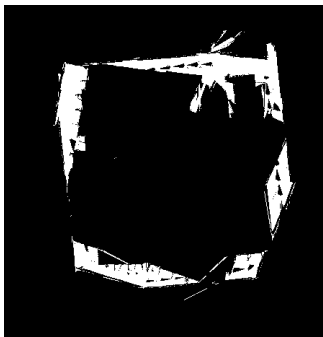


Fig. 15. Matching map - before odometry calibration

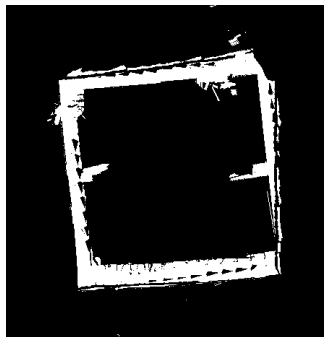


Fig. 16. Matching map - after odometry calibration

With this new comparison map created, we then developed a script that counts the number of free/occupied cells in the Gmapping-created map and number of white cells in the matching map. This was done with a simple pixel-counting function that returns the number of cells for each colour. We used this analysis for the map generated using pure odometry and the one using corrected odometry.

TABLE I

Map	Occupied cells	Free cells
GMapping	46 725	160 683
Matching w/o odom calib.	18 824	188 584
Matching w/ odom calib.	23 650	183 758

After analysing the results, we observed that the map generated before odometry calibration matched approximately **40%** of the ground truth (Gmapping), while the map created after odometry calibration reached a **51%** match, which reflects a strong structural alignment between the two maps. This improvement is not only quantitatively significant, with an

11 percentage increase, but also visually evident, this part is especially evident on corridors and corners. We can also say that the matching map with odometry calibration has a clear and coherent layout, suggesting that the robot's perception of free and occupied space is consistent with the actual environment.

We consider that the mapping process was overall successful. Our best map shows a clear improvement over the one based on raw odometry, both in terms of structure and in overlap with the ground truth. Although some mismatches are expected, especially in areas with sparse data or angular drift, we consider that the general structure aligns well with the ground truth.

TABLE II

Method	IoU (%)	Notes
Simulator (low-noise)	83	Case without high values of pose or sensor noise.
Simulator (high-noise)	46	Includes odometry drift and sensor noise.
Real – Uncalibrated odometry	40	Angular drift causes trajectory distortion.
Real – Calibrated odometry	51	Improved loop closure and map alignment.

VI. CONCLUSIONS

In this project, we implemented and evaluated an occupancy grid mapping system using a Pioneer 3DX equipped with a Hokuyo laser. To better understand the behavior of our algorithm, we also developed a simulation environment that allowed controlled testing with realistic sensor noise and motion uncertainty. This provided valuable insight and enabled us to draw comparisons between simulated and real-world scenarios.

After implementing our code, and extracting bags with data from the 5th floor we focused on analyzing the resulting maps, and improving their quality, not only in terms of the code used for the mapping, but also exploring different technics to correct the pose of the robot, such as calibrating the pioneer and experimenting with AMCL (even though it it provided really good angular it resulted in poorer overall quality). In the end, and in comparison with the ground truth map, getting the pose using well calibrated odometry allowed us to generate more stable trajectories and sharper, more coherent maps, making this technic the most reliable and consistent.

Overall, we consider that our project successfully demonstrates the potential of occupancy grid mapping when supported by suitable localization strategies and careful tuning of motion parameters.

REFERENCES

- [1] S. Thrun, W. Burgard, and D. Fox, *Probabilistic Robotics*, MIT Press, 2005.
- [2] ROS Wiki, *tf – Using tf with Python*. Available at: <https://wiki.ros.org/tf/TfUsingPython>. Accessed on 30 May 2025.
- [3] Pioneer 3 Operations Manual: MobileRobots Inc, 2006.